Evan Edelstein
EN.605.645.82.SP26
Module 7 – self check
Pseudocode

```
1 def unification( exp1, exp2):
2     # base cases
3     if exp1 and exp2 are constants or the empty list:
4         if exp1 = exp2 then return {}
5         else return FAIL
6
7     if exp1 is a variable:
8         if exp1 occurs in exp2 then return FAIL
9         else return {exp1/exp2}
10
11    if exp2 is a variable:
12        if exp2 occurs in exp1 then return FAIL
13        else return {exp2/exp1}
14
15    # inductive step
16    first1 = first element of exp1
17    first2 = first element of exp2
18
19    result1 = unification( first1, first2)
20    if result1 = FAIL then return FAIL
21
22    apply result1 to rest of exp1 and exp2
23
24    result2 = unification( rest of exp1, rest of exp2)
25    if result2 = FAIL then return FAIL
26    return composition of result1 and result2
```

Expression 1	***Expression 2***	**Result**
Fred	Barney	FAIL
Pebbles	Pebbles	{}
(quarry_worker Fred)	(quarry_worker ?x)	{?x/Fred
(son Barney ?x)	(son ?y Bam_Bam)	{?y/Barney, ?x/Bam_Bam}
(married ?x ?y)	(married Barney Wilma)	{?x/Barney, ?y/Wilma}
(son Barney ?x)	(son ?y (son Barney))	{?y/Barney, {?x/(son Barney))
(son Barney ?x)	(son ?y (son ?y))	{?y/Barney, {?x/(son Barney))}
(son Barney Bam_Bam)	(son ?y (son Barney))	FAIL
(loves Fred Fred)	(loves ?x ?x)	{?x/Fred}
(future George Fred)	(future ?y ?y)	FAIL

a. Unify(Fred, Barney): We can see from line 3-5 in the pseudocode that if both inputs are constants and not equal, unify returns FAIL

b. Unify(Pebbles, Pebbles): We can see from line 3-5 in the pseudocode that if both inputs are constants and are equal, unify returns an empty substitution list.
c. Unify((quarry_worker Fred), (quarry_worker, ?x)): Since both arguments are expressions we hit the inductive step, and extract quarry_worker from both arguments. The recursive unification returns an empty substitution list since the inputs are equal constants. Since it is empty, there is nothing to apply and we call unify again with Fred and ?x. Since the second input is a variable and does not occur in the first ,we return a substation list containing {?x/Fred}.
d. Unify((son Barney ?x) (son ?y Bam_Bam)). Following the pseudocode, we extract son and son and return an empty substitution list. Next, we extract Barney and ?y and return a substitution list of {?y/Barney}. Lastly, we recursively call unify(?x, Bam_Bam) and return {?x/Bam_Bam}. Combining the two substitution lists we get {?y/Barney, ?x/Bam_Bam}.
e. Unify((married ?x ?y), (married Barney Wilma)). Trivially we can see that we can assign each constant in the second argument to the two variables in the first, returning {?x/Barney, ?y/Wilma}
f. [also 2] Unify((son Barney ?x), (son ?y (son Barney))). Following the pseudocode, we pop son and son off the inputs and recursively return an empty substitution list. Recursively, we then pop off Barney and ?y and get the substitution list {?y/Barney}. Next, we recursively call unify(?x, (son Barney)). Since the first input is a variable and it does not occur in the second input, by line 9, we add {?x/(son Barney)} to the substitution list. This results in the solution {?y/Barney, ?x/(son Barney))}.
g. Unify((son Barney ?x), (son ?y (son ?y))). We follow the same logic as (f), but at the application step we change (son ?y) to (son Barney), this then returns the same solution {?y/Barney, {?x/(son Barney))}.
h. Unify((son Barney Bam_Bam), (son ?y (son Barney))). Following the above steps we get the substation list {?y/Barney}. Next, we recursively call unify(Bam_Bam, (son Barny)). Since we cannot assign a constant function to a constant we return Fail as the final solution.
i. Unify((loves Fred Fred), (loves ?x ?x)): trivially we have the substitution list {?x/Fred}
j. Unify((loves Geroge Fred), (loves ?y ?y)): Trivially we cannot assign the same variable to two different constants, so this returns FAIL.

2: see (f)